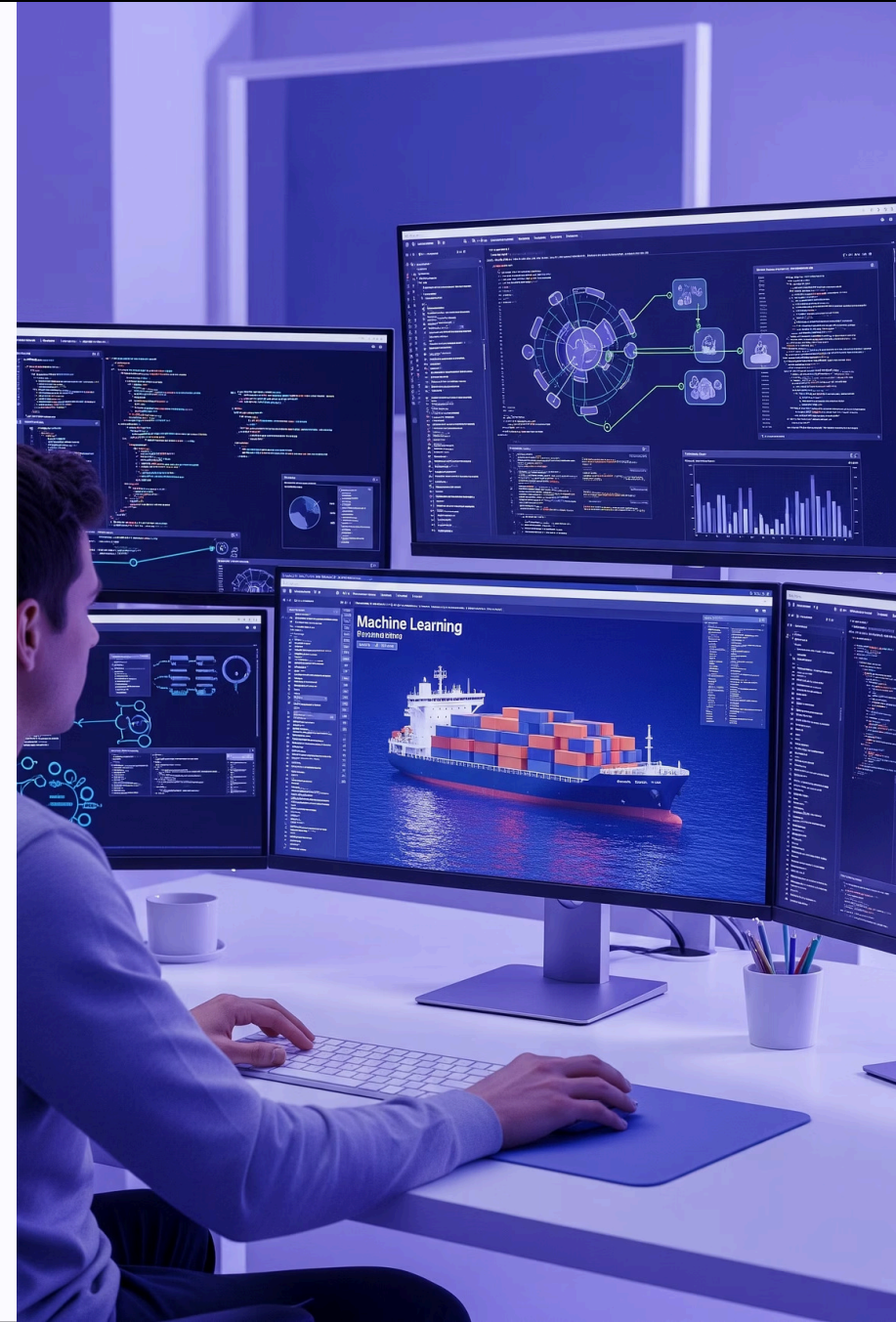


Building a Container for an AI App

A practical guide to containerizing a simple AI application for seamless deployment across any environment.



Why Containerize an AI App?

Portability

Package ML models + code into a single portable unit that runs consistently anywhere

Deployment Simplicity

Streamlines deployment processes and enables easy scaling

Reproducibility

Guarantees identical environments across development and production

API Foundation

Creates the foundation for serving AI models as production-ready APIs



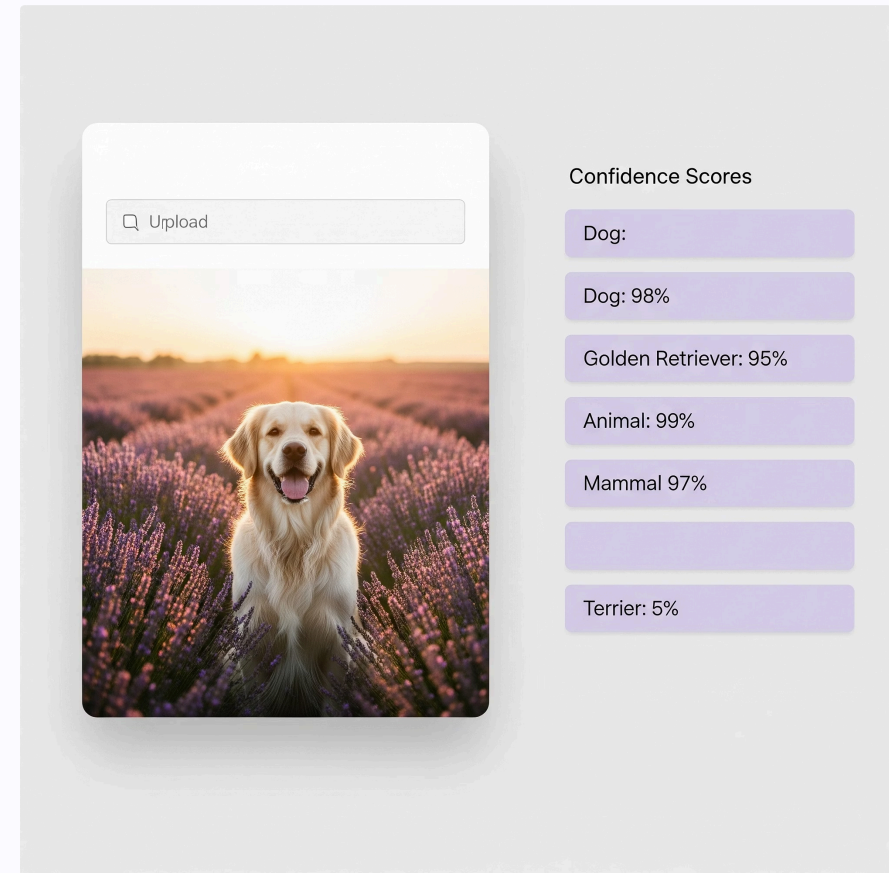
Example AI App – Image Classifier

What We're Building

A containerized image classification service that:

- Accepts image uploads from users
- Processes them with a pre-trained ResNet18 CNN model
- Returns predicted labels (dog, cat, etc.)
- Runs in a FastAPI server

This simple yet powerful example demonstrates all key containerization principles.



Project Structure

```
ai-app/  
├── app.py      # FastAPI code  
├── model.pt    # saved PyTorch  
model  
├── requirements.txt  
└── Dockerfile
```

Keep your AI app self-contained with all necessary components in a single directory for easy containerization.

Design Principles

Self-Contained

Everything the app needs is in one directory

Versioned Dependencies

Requirements specify exact package versions

Container Blueprint

Dockerfile defines the build process

The requirements.txt

```
fastapi==0.103.0
uvicorn==0.23.2
torch==2.1.0
torchvision==0.16.0
pillow==10.0.0
```

Dependency Management

Explicitly defines all Python packages needed to run the application

Version Pinning

Specifies exact versions to ensure consistent behavior across environments

Reproducibility

Guarantees that anyone building the container gets identical dependencies

Pro tip: For ML applications, always pin exact versions to avoid subtle model behavior changes when packages update.

Writing app.py

```
from fastapi import FastAPI, UploadFile
from PIL import Image
import torch, torchvision.transforms as T

app = FastAPI()
model = torch.load("model.pt")
model.eval()
transform = T.Compose([T.Resize((224,224)), T.ToTensor()])

@app.post("/predict")
async def predict(file: UploadFile):
    img = Image.open(file.file)
    x = transform(img).unsqueeze(0)
    y = model(x)
    return {"prediction": y.argmax().item()}
```

The API Logic

- Load pre-trained PyTorch model
- Set up image transformation pipeline
- Create endpoint for image prediction

Key Features

- Asynchronous handler for better performance
- Simple JSON response format
- Minimal code for maximum clarity

The Dockerfile

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```



Base Image

Start with lightweight Python image



Dependencies

Install requirements first (for better caching)



Application Code

Copy app code and model file



Launch Command

Start FastAPI server on container boot

Build the Image



```
docker build -t ai-app:latest .
```

What's Happening

- `-t ai-app:latest` assigns a name and tag
- The `.` specifies current directory as build context
- Each Dockerfile instruction creates a layer
- The result is a portable container image

Verify with `docker images` to see your new AI app image.

Remember: The build process may take several minutes the first time as PyTorch and its dependencies are downloaded.

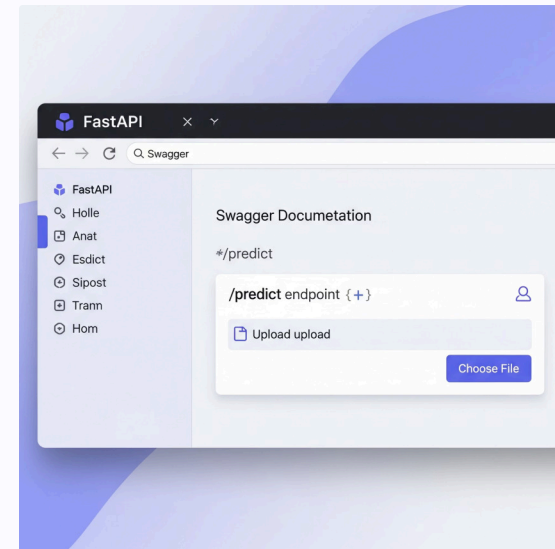
Run the Container

```
docker run -it -p 8000:8000 ai-app:latest
```

Command Breakdown

- `-it` runs interactively with a terminal
- `-p 8000:8000` maps container port to host port
- `ai-app:latest` specifies the image to run

Your AI app is now accessible at <http://localhost:8000/docs>

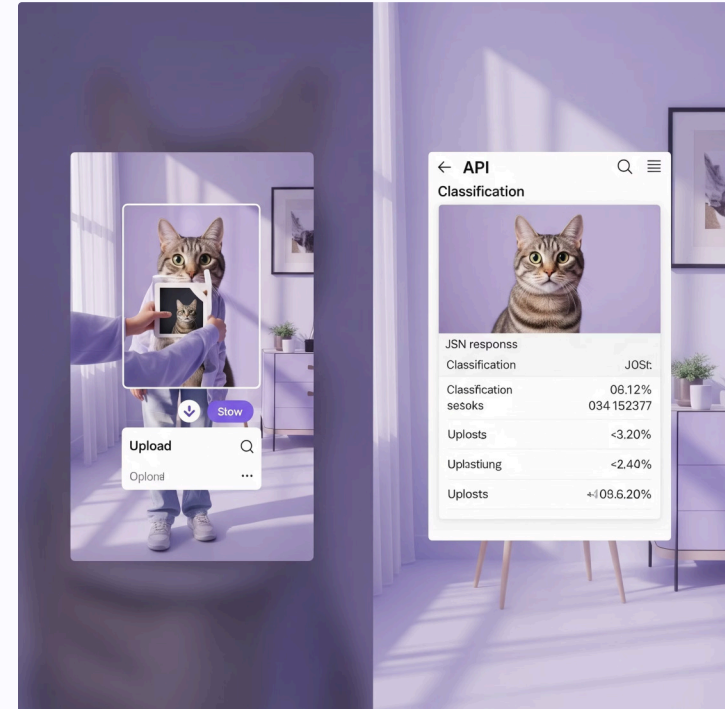


- ❗ The container isolates the app but allows controlled access through the mapped port. Users interact with it as if it were running directly on the host.

Testing the API

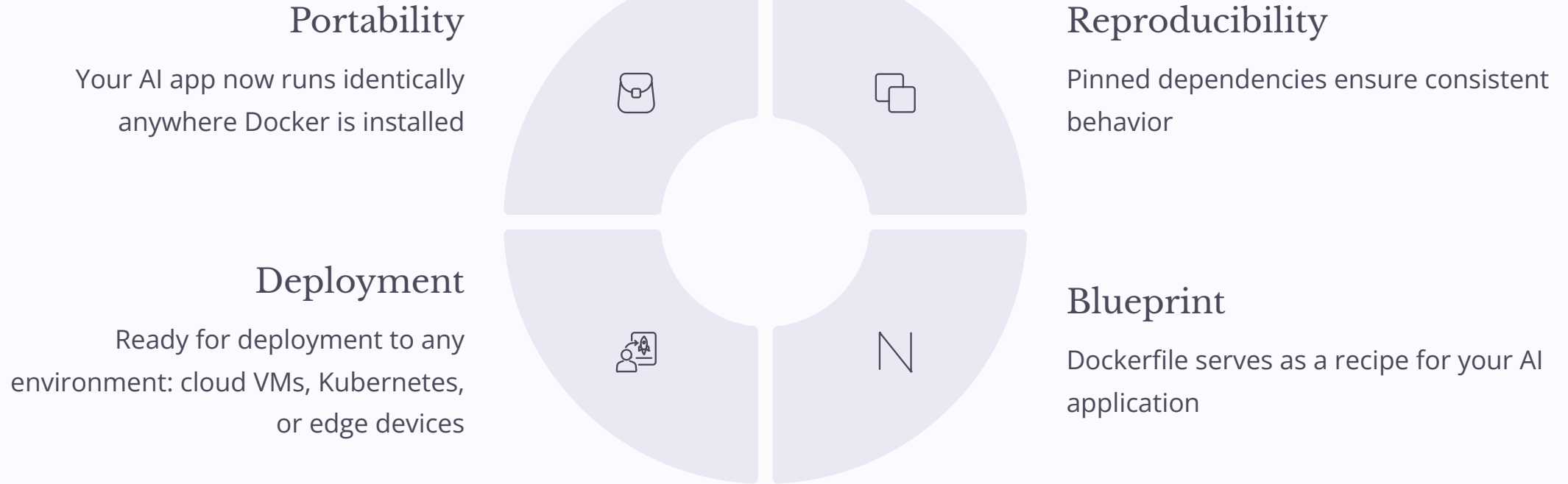
How to Test

1. Navigate to <http://localhost:8000/docs>
2. Click the `/predict` endpoint to expand it
3. Click "Try it out" and upload an image
4. Execute the request and view prediction results
5. Check container logs in your terminal



✅ A successful test confirms that your ML model is properly containerized and serving predictions through the API.

Key Takeaways



Containerization is a fundamental skill for bringing ML models to production

Next steps: Add monitoring, CI/CD pipelines, and explore orchestration with Kubernetes